

Aprenentatge Automàtic II: Fonaments de l'Aprenentatge Automàtic i Deep Learning

Codi: 104871 - Grau en Estadística Aplicada

Víctor Navas Portella

Segon quadrimestre- Curs 2025/2026

UAB
Universitat Autònoma
de Barcelona

- 1 Gradient descent fitting models
 - Stochastic Gradient Descent
- 2 Backpropagation: Càlcul de les derivades en una xarxa
- 3 Inicialització de paràmetres
- 4 Mètriques de performance
- 5 Regularització

Entrenament: L'Objectiu

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

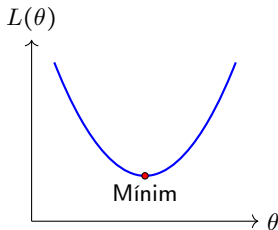
Mètriques de
performance

Regularització

Ja sabem com la informació viatja cap endavant (**Feed-forward**) la informació en una xarxa. Però, com trobem els pesos Θ i biaixos θ_0 "bons"?

- 1 **Definir l'error:** Usem una funció de pèrdua $L(\hat{y}, y)$ (MSE, BCE, CCE).
- 2 **Cercar el mínim:** Volem els paràmetres $(\theta = \{\Theta^{(l)}, \theta_0^{(l)}\})$, amb $l = 1, \dots, K$) pesos i biaixos que minimitzin aquesta pèrdua:

$$\theta^* = \arg \min_{\theta} L(\theta)$$



L'algorisme per moure'ns cap al mínim és el **Gradient Descent**.

Un context amable: Regressió lineal simple

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

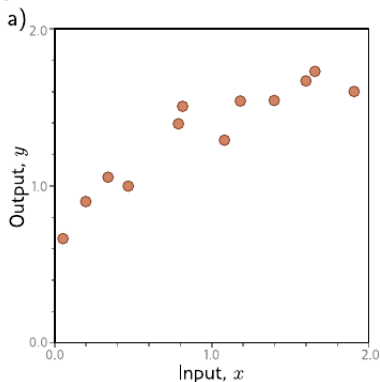
Mètriques de
performance

Regularització

En una regressió lineal simple $\hat{y} = \phi_1 x + \phi_0$ (ϕ_1 seria el pes i ϕ_0 el biaix), busquem els paràmetres de la recta que millor s'ajusta a les dades minimitzant la loss function del **Mean Squared Error**:

$$L(\phi_1, \phi_0) = \frac{1}{2} \frac{1}{n} \sum_{i=1}^n (y_i - (\phi_1 x_i + \phi_0))^2$$

En lloc de resoldre-ho analíticament (equacions normals), com ho faria un algorisme si només pogués “sentir” el pendent sota els seus peus? **Això és el Gradient Descent.**



Descens del gradient en regressió lineal simple

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Per a la regressió lineal $\hat{y} = \phi_1 x + \phi_0$, definim el vector de paràmetres com $\theta = \begin{bmatrix} \phi_1 \\ \phi_0 \end{bmatrix}$. L'actualització es realitza seguint la geometria del gradient:

1. Desglòs del vector Gradient El gradient $\nabla L(\theta)$ és el vector de les derivades parcials de la funció de pèrdua:

$$\nabla L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial \phi_1} \\ \frac{\partial L}{\partial \phi_0} \end{bmatrix} = \begin{bmatrix} -\frac{1}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i) \\ -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \end{bmatrix}$$

2. Actualització component a component L'expressió $\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$ es tradueix en:

$$\begin{bmatrix} \phi_1 \\ \phi_0 \end{bmatrix}_{t+1} = \begin{bmatrix} \phi_1 \\ \phi_0 \end{bmatrix}_t - \eta \cdot \begin{bmatrix} \frac{\partial L}{\partial \phi_1} \\ \frac{\partial L}{\partial \phi_0} \end{bmatrix}$$

D'on obtenim les regles individuals (regla de Delta):

- $\phi_1^{(t+1)} = \phi_1^{(t)} + \eta \cdot \frac{1}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i)$
- $\phi_0^{(t+1)} = \phi_0^{(t)} + \eta \cdot \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$

Descens del gradient en un cas senzill

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

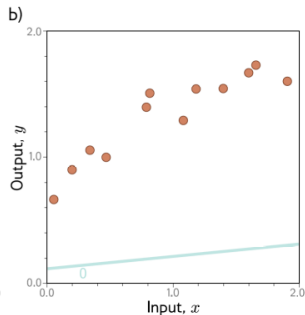
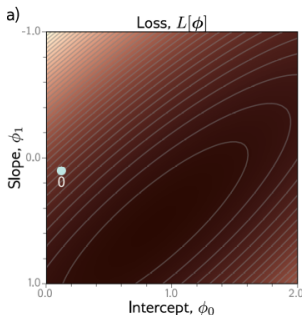
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Matemàticament, estem actualitzant el vector de paràmetres $\theta = [\phi_1, \phi_0]^T$:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

Les claus del procés:

- **El Gradient (∇L):** Ens indica la direcció de màxim creixement. Per això restem, per anar en direcció contrària (avall).
- **El pas (η):** Controla quant "saltem" en cada iteració.
- **Convergència:** A mesura que ens acostem al mínim, el gradient es fa petit i els canvis en la recta són gairebé imperceptibles.

Descens del gradient en un cas senzill

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

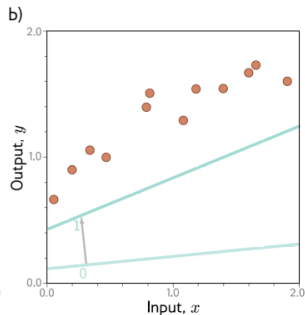
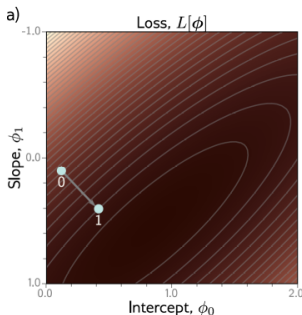
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Matemàticament, estem actualitzant el vector de paràmetres $\theta = [\phi_1, \phi_0]^T$:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

Les claus del procés:

- **El Gradient (∇L):** Ens indica la direcció de màxim creixement. Per això restem, per anar en direcció contrària (avall).
- **El pas (η):** Controla quant "saltem" en cada iteració.
- **Convergència:** A mesura que ens acostem al mínim, el gradient es fa petit i els canvis en la recta són gairebé imperceptibles.

Descens del gradient en un cas senzill

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

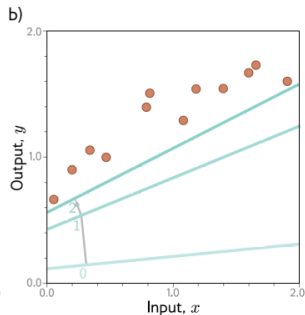
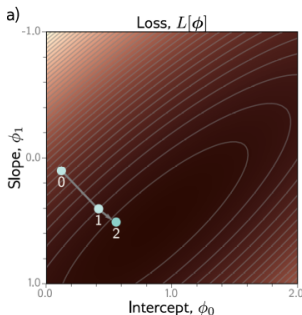
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Matemàticament, estem actualitzant el vector de paràmetres $\theta = [\phi_1, \phi_0]^T$:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

Les claus del procés:

- **El Gradient (∇L):** Ens indica la direcció de màxim creixement. Per això restem, per anar en direcció contrària (avall).
- **El pas (η):** Controla quant "saltem" en cada iteració.
- **Convergència:** A mesura que ens acostem al mínim, el gradient es fa petit i els canvis en la recta són gairebé imperceptibles.

Descens del gradient en un cas senzill

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

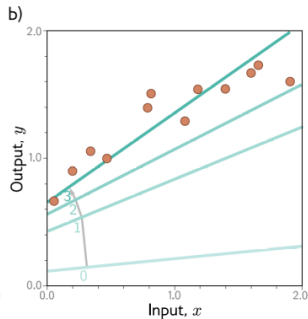
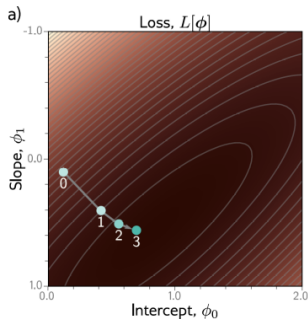
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Matemàticament, estem actualitzant el vector de paràmetres $\theta = [\phi_1, \phi_0]^T$:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

Les claus del procés:

- **El Gradient (∇L):** Ens indica la direcció de màxim creixement. Per això restem, per anar en direcció contrària (avall).
- **El pas (η):** Controla quant "saltem" en cada iteració.
- **Convergència:** A mesura que ens acostem al mínim, el gradient es fa petit i els canvis en la recta són gairebé imperceptibles.

Descens del gradient en un cas senzill

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

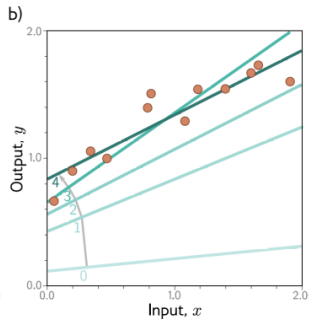
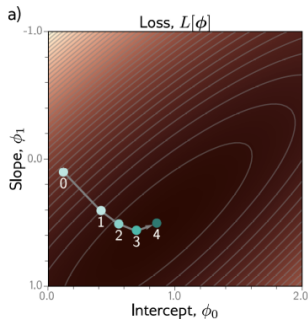
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Matemàticament, estem actualitzant el vector de paràmetres $\theta = [\phi_1, \phi_0]^T$:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

Les claus del procés:

- **El Gradient (∇L):** Ens indica la direcció de màxim creixement. Per això restem, per anar en direcció contrària (avall).
- **El pas (η):** Controla quant "saltem" en cada iteració.
- **Convergència:** A mesura que ens acostem al mínim, el gradient es fa petit i els canvis en la recta són gairebé imperceptibles.

Optimització de Segon Ordre: La Matriu Hessiana

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Si $L(\theta)$ és la funció de pèrdua, el **Gradient** ens dona la direcció de màxima variació, però la **Hessiana** ens dona la **curvatura**. La Hessiana $\mathbf{H} = \nabla^2 L(\theta)$ és la matriu de segones derivades parcials:

$$H_{ij} = \frac{\partial^2 L}{\partial \theta_i \partial \theta_j}$$

Convexitat i optimització

La naturalesa del punt crític ($\nabla L = 0$) es determina analitzant el signe de la Hessiana (forma quadràtica) per a qualsevol vector de desplaçament $\mathbf{v} \neq 0$. L'escalar $q(\mathbf{v}) = \mathbf{v}^T \mathbf{H} \mathbf{v}$ defineix la curvatura en la direcció $\mathbf{v}$:

- **Mínim Estricte:** $\mathbf{H}$ és **Definida Positiva** ($\lambda_i > 0, \forall i$). Curvatura positiva en totes les direccions.
- **Màxim Estricte:** $\mathbf{H}$ és **Definida Negativa** ($\lambda_i < 0, \forall i$). Curvatura negativa en totes les direccions.
- **Punt de Sella:** $\mathbf{H}$ és **Indefinida** (λ_i amb signes oposats). La funció puja en unes direccions i baixa en d'altres.
- **Semidefinida Positiva** ($\lambda_i \geq 0$): Candidat a mínim local.
- **Semidefinida Negativa** ($\lambda_i \leq 0$): Candidat a màxim local.

Hessiana i convexitat en regressió lineal simple

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Recuperem la Hessiana obtinguda per a la regressió lineal simple:

$$\mathbf{H} = \frac{1}{n} \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix} = \frac{1}{n} \mathbf{X}^T \mathbf{X}, \quad \text{amb } \mathbf{X} = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}$$

Exercici

Demostreu que la Hessiana:

$$\mathbf{H} = \frac{1}{n} \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix} = \frac{1}{n} \begin{bmatrix} s_{xx} & s_x \\ s_x & n \end{bmatrix},$$

és definida positiva

L'Optimització de Newton i el cost del segon ordre

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Si poguéssim usar la Hessiana, faríem servir el **Mètode de Newton**. En lloc de seguir només el pendent, corregiríem el pas segons la curvatura local:

Regla d'actualització de Newton

$$\theta_{t+1} = \theta_t - \eta \mathbf{H}^{-1} \nabla L(\theta_t)$$

Per què no ho fem en Xarxes Neuronals?

- **Inversió de la matriu:** Invertir una matriu de mida $N \times N$ té un cost computacional de $O(N^3)$. Amb milions de paràmetres, és totalment inviable.
- **El problema de la dimensionalitat:** Per a un model amb $N = 10^6$ paràmetres, $\mathbf{H}$ té 10^{12} elements. Emmagatzemar-la requeriria aprox. **4 TB de RAM** (en *float32*).
- **Punts de sella (*Saddle points*):** En espais d'alta dimensió, la pèrdua està plena de punts de sella. El mètode de Newton se sent atret per ells si la Hessiana no és definida positiva, a diferència del Gradient Descent.

Malgrat el Gradient Descent (primer ordre) és una aproximació “més pobre”, és molt més **escalable** i **robusta** davant de curvatures complexes.

El model de Gabor

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

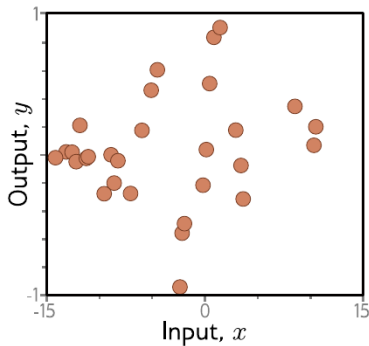
Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Un exemple senzill però menys amable d'un model amb dos paràmetres $\theta = \{\phi_0, \phi_1\}$ és el model de Gabor:

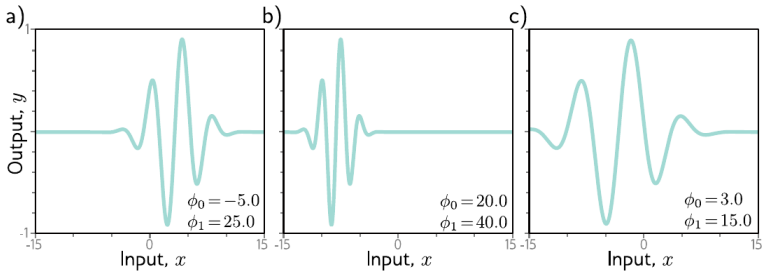
$$f(x; \phi_0, \phi_1) = \sin(\phi_0 + a\phi_1 x) \exp\left(-\frac{(\phi_0 + a\phi_1 x)^2}{b}\right), \quad \text{amb } a = 0.06 \text{ i } b = 32.$$



El model de Gabor

Un exemple senzill però menys amable d'un model amb dos paràmetres $\theta = \{\phi_0, \phi_1\}$ és el model de Gabor:

$$f(x; \phi_0, \phi_1) = \sin(\phi_0 + a\phi_1 x) \exp\left(-\frac{(\phi_0 + a\phi_1 x)^2}{b}\right), \quad \text{amb } a = 0.06 \text{ i } b = 32.$$



Exercici

- Calcula les primeres derivades respecte ϕ_0 i ϕ_1 de la Loss function L (MSE).
- Troba la Hessiana de L i discuteix la diversitat de signes dels valors propis.

Loss function del model de Gabor

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

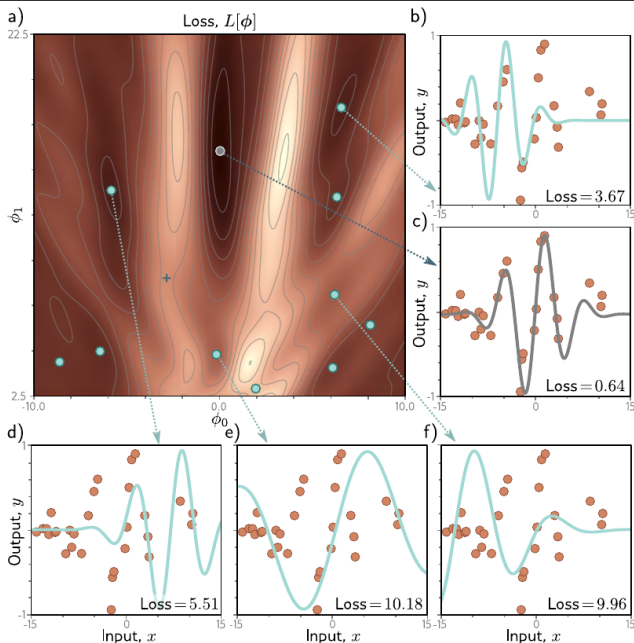
Stochastic Gradient Descent

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Debat: Convexitat en Deep Learning

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Es pot garantir la convexitat de la funció de pèrdua $L(\theta)$ en una xarxa neuronal?

- Les xarxes són **altament no convexes** a causa de les activacions no lineals i la multiplicació de paràmetres (capes ocultes).
- **Punts de Sella (*Saddle Points*)**: En espais de moltes dimensions, el gradient zero sol correspondre a punts on la Hessiana té valors propis positius i negatius alhora.

Per tant, tenim doble dificultat:

- 1 El Mur Computacional
- 2 El Mur de la No-Convexitat

La Gran Pregunta

Com podem navegar de manera eficient per una hipersuperfície d'un milió de dimensions sense quedar-nos atrapats en zones de gradient gairebé nul?

Necessitem un mètode que:

- Només requereixi informació de primer ordre (gradients).
- Sigui capaç d'introduir "dinamisme" per no aturar-se en punts de sella.

Gradient Descent: Baixant la muntanya

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Si estem en un punt de la funció de pèrdua, el gradient $\nabla_{\theta}L$ ens indica la direcció de **màxim creixement**. Per tant, hem d'anar en direcció contrària.

Regla d'actualització

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta}L$$

On:

- η : El *learning rate* (mida del pas).
- $\nabla_{\theta}L$: El vector de derivades parcials respecte tots els paràmetres.

El problema: Com calculem les derivades $\frac{\partial L}{\partial \Theta^{(l)}}$ en una xarxa amb moltes capes on els paràmetres estan "amagats" a l'interior?

La solució: L'algorisme de **Backpropagation**.

El problema del Gradient Descent "Vanilla"

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Stochastic Gradient Descent

Backpropagation: Càlcul de les derivades en una xarxa

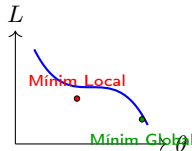
Inicialització de paràmetres

Mètriques de performance

Regularització

Fins ara hem vist el **Batch Gradient Descent** (BGD), on calculem el gradient usant **tot el conjunt d'entrenament** ($\mathcal{D}_{train}$) abans de fer un sol pas d'actualització.

- **MNIST:** 60,000 imatges $\times$ 84,060 paràmetres $\rightarrow$ Càlcul massiu només per moure els pesos *una* vegada!
- **Memòria:** No podem carregar datasets de GigaBytes (com a imatges mèdiques o vídeos) a la vegada a la memòria de la GPU (VRAM).
- **Convergència:** En superfícies d'error complexes, el gradient "exacte" del Batch pot portar-nos directament a un **mínim local** o quedar-se estancat en un **punt de sella** (saddle point).



Nota metodològica

Recordeu: el gradient es calcula **només** sobre les dades d'entrenament. Les dades de test mai s'utilitzen per actualitzar els paràmetres.

Stochastic Gradient Descent (SGD)

En lloc d'esperar a processar tot $\mathcal{D}_{train}$, actualitzem els paràmetres després de cada **exemple individual** (x_i, y_i) .

Algorisme SGD (Online)

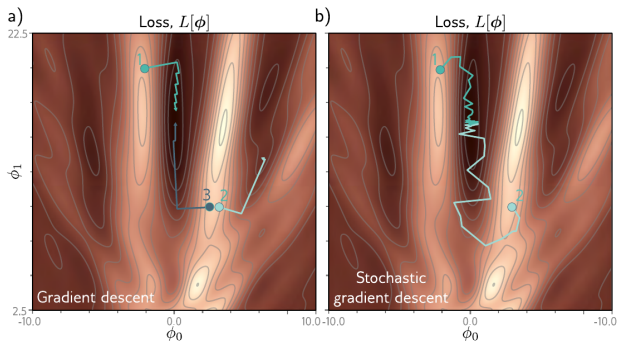
Per a cada època:

- ➊ **Shuffle**: Desordenar les dades per evitar biaixos d'ordre.
- ➋ Per a cada parella $(x_i, y_i) \in \mathcal{D}_{train}$:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L(\theta; x_i, y_i)$$

Característiques:

- **Velocitat**: Fem N actualitzacions (tants com exemples individuals).
- **Soroll**: El gradient d'un sol exemple és una estimació "sorollosa" del gradient real.
- **Exploració**: Aquest soroll permet a la funció de pèrdua "saltar" i escapar de mínims locals poc profunds.



- A la figura a) (BGD), si els paràmetres inicials s'inicialitzen als punts 1 o 3, acabarem al mínim global. No ho farà en canvi si parteix del punt 2.
- A la figura b) (SGD), tant si partim de 1 o de 2, acabarem al mínim global perquè el soroll ens fa escapar de mínims locals o punts de sella.

L'estàndard: Mini-Batch SGD

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Stochastic Gradient Descent

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

A la pràctica, busquem un compromís entre l'estabilitat del Batch GD i la velocitat del SGD pur fent servir un **Mini-Batch** de mida B .

Conceptes Clau

- **Batch Size (B):** Nombre d'exemples per actualització (típicament 32, 64, 128 o 256).
- **Iteració (Step):** Un únic pas d'actualització de pesos usant un mini-batch.
- **Època (Epoch):** Una passada completa per tots els exemples de $\mathcal{D}_{train}$.

Relació matemàtica

Si tenim N dades totals al training set:

$$\text{Iteracions per època} = \left\lceil \frac{N}{B} \right\rceil$$

Per què funciona millor? Permet aprofitar la **vectorització** de les GPUs (processar 32 imatges és gairebé tan ràpid com processar-ne 1) i redueix la variància del gradient respecte al SGD pur.

Mini-Batch SGD: L'estàndard de la indústria

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Victor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

A la pràctica, no usem ni 1 exemple ni tot el dataset, sinó un **Mini-Batch** (típicament de 32, 64 o 128 exemples).

Mètode	Exemples/pas	Estabilitat	Velocitat
Batch GD	Tot el dataset	Molta (exacte)	Molt lent
SGD	1 exemple	Molt sorollós	Molt ràpid
Mini-Batch	8 – 256	Bona	Optimitzat GPU

Per què 32 o 64?

Les matrius $\Theta^{(l)}$ es calculen en paral·lel a la GPU. Processar 32 imatges a la vegada és gairebé tan ràpid com processar-ne una sola gràcies a la vectorització.

Exercici: Quants cops aprenem?

Volem entrenar el nostre model MNIST (60,000 imatges) durant **10 èpoques**. Calcula el nombre d'actualitzacions de pesos segons el mètode:

❶ **Batch Gradient Descent:**

- 1 actualització per època $\times$ 10 èpoques = **10 actualitzacions**.

❷ **Stochastic Gradient Descent (Pur):**

- 60,000 actualitzacions per època $\times$ 10 èpoques = **600,000 actualitzacions**.

❸ **Mini-Batch SGD (mida de batch = 100):**

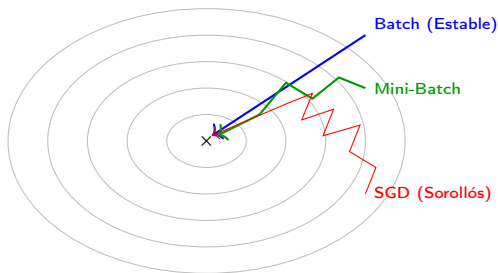
- Pas 1: Iteracions per època = $60,000/100 = 600$.
- Pas 2: 600 iteracions $\times$ 10 èpoques = **6,000 actualitzacions**.

Conclusió

El Mini-Batch SGD ens dona el millor de dos mons: l'estabilitat del Batch i la velocitat de l'Stochastic.

Conclusions visualitzades:

- **Batch** és el camí més directe però molt costós de calcular.
- **SGD** és erràtic, però cada pas és molt "barat".
- **Mini-Batch** és el camí òptim: prou estable per convergir i prou ràpid per aprofitar el hardware.



Limitacions del SGD i Variants

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

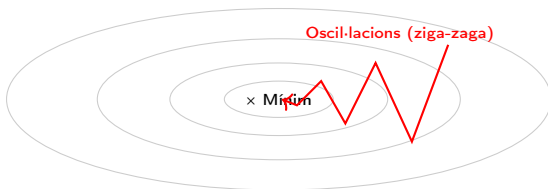
Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Tot i que el Mini-Batch SGD és potent, s'enfronta a geometries de la funció de pèrdua que dificulten la convergència:

- 1 **Valls Estretes (Poor Scaling):** Si la superfície d'error és molt pendent en una dimensió però plana en una altra, el SGD oscil·la violentament a les "parets" de la vall, avançant molt poc cap al mínim.
- 2 **Punts de Sella (Saddle Points):** En alta dimensió, els punts on el gradient és zero solen ser punts de sella (mínims en una direcció, màxims en una altra). El SGD pot quedar-se estancat on el pendent és gairebé nul.



El problema de la curvatura

El gradient ens diu la direcció de màxim pendent **local**, però no té "memòria" ni informació sobre la curvatura global de la funció.

Gradient descent meme

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
nentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

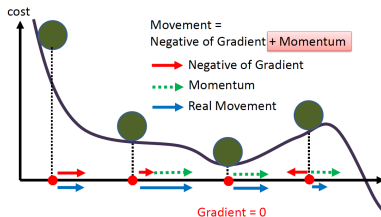
Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització





El SGD s'atura quan $\nabla L = 0$. Introduint **inèrcia**, fem que l'actualització recordi la velocitat prèvia, permetent "superar" petits turons o zones planes de la funció de pèrdua.

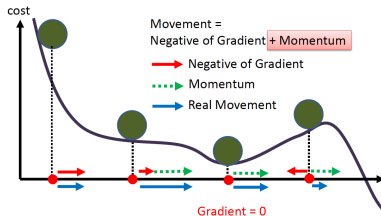
Regla d'actualització i Mitjana Mòbil

El momentum $\mathbf{m}_t$ s'actualitza com una **mitjana mòbil exponencial (EMA)** dels gradients passats:

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + (1 - \beta) \nabla_{\theta} L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{m}_{t+1}$$

Momentum II



Per què és una suma exponencial? Si expandim la recurrència, veiem que el pes de cada gradient disminueix geomètricament cap al passat:

$$\mathbf{m}_{t+1} = (1 - \beta) \sum_{r=0}^t \beta^{t-r} \nabla_{\theta} L(\theta_t)$$

- $\beta \in [0, 1)$: Controla la "memòria". Un $\beta = 0.9$ equival a fer una mitjana dels darrers ≈ 10 gradients.
- Guanyem **estabilitat**: el terme $\mathbf{m}$ suavitza les variacions brusques de cada mini-batch individual.

El moviment real és la suma del vector gradient (vermell) i la inèrcia acumulada (verd). Això ens permet no quedar-nos atrapats quan el gradient és nul.

Nesterov Accelerated Gradient (NAG)

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Stochastic Gradient Descent

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

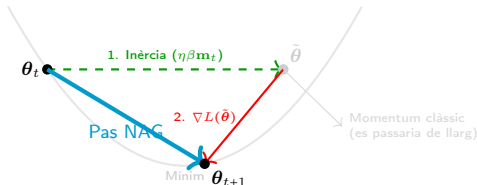
El Momentum clàssic calcula el gradient en el punt actual. El NAG primer es deixa portar per la inèrcia i, **des de la posició futura**, calcula la correcció.

L'estratègia de Nesterov

- 1 Salt d'inèrcia: Posició provisional $\tilde{\theta} = \theta_t - \eta\beta\mathbf{m}_t$.
- 2 Correcció: Calculem el gradient en aquest nou punt $\nabla L(\tilde{\theta})$.

$$\mathbf{m}_{t+1} = \beta\mathbf{m}_t + (1 - \beta)\nabla_{\theta}L(\tilde{\theta})$$

$$\theta_{t+1} = \theta_t - \eta\mathbf{m}_{t+1}$$



Per què és millor? Si el salt d'inèrcia ens porta a l'altra paret de la vall, el gradient a $\tilde{\theta}$ ens indica immediatament que hem de frenar, evitant l'*overshooting*.

RMSProp: Adaptant-se a la curvatura

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Mentre que el Momentum actua sobre la *direcció*, **RMSProp** ajusta el **pas** (η efectiva) per a cada paràmetre de forma individual.

- 1 **Estimació del Segon Moment (Variabilitat):** Mantenim una mitjana mòbil del quadrat del gradient per capturar la seva magnitud típica o "energialocal":

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2)(\nabla_{\theta} L(\boldsymbol{\theta}_t))^2$$

Estadísticament: $\mathbf{v}_{t+1}$ estima el segon moment no centrat (relacionat amb la variància).

- 2 **Actualització (Normalització de la variància):** Dividim el gradient per la seva "desviació típica"aproximada ($\sqrt{\mathbf{v}_{t+1}}$):

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla_{\theta} L(\boldsymbol{\theta}_t)$$

Els hiperparàmetres $\beta_2 \approx 0.99$ (memòria de l'escala), $\epsilon \approx 10^{-8}$ (estabilitat numèrica).

RMSPProp: Per què funciona?

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
ntatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

- **Gradients amb molta variància:** El denominador creix $\rightarrow$ el pas es fa més petit i **prudent** (frenem l'oscil·lació).
- **Gradients petits i constants:** El denominador és petit $\rightarrow$ el pas s'amplifica i **accelerem** en zones planes.
- **Estandardització:** El gradient esdevé una variable de "escala unitària" (*unit scaling*), fent l'aprenentatge robust a l'escala de la funció de pèrdua.

Imagina una vall que és molt estreta en la direcció y (gradients grans) i molt plana en la direcció x (gradients petits):

- **En la direcció vertical (y):** Els gradients són grans, per tant $\mathbf{v}_t$ creix. Com que dividim per $\sqrt{\mathbf{v}_t}$, el pas es fa **més petit**, evitant que el model surti disparat de la vall.
- **En la direcció horitzontal (x):** Els gradients són minúsculs, per tant $\mathbf{v}_t$ és petit. Dividir per un número petit **amplifica el pas**, fent que avancem més ràpid cap al mínim.

SGD: Oscil·la sense avançar



Adam: Adaptive Moment Estimation

Aprenentatge
Automàtic II:
Fonaments
de l'Apren-
entatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Adam (Adaptive Moment Estimation) combina dues idees clau:

- ➊ **Momentum**: Manté una mitjana mòbil dels gradients ($\mathbf{m}_t$).
- ➋ **RMSProp**: Manté una mitjana mòbil del **quadrat** dels gradients ($\mathbf{v}_t$) per adaptar el *learning rate* a cada paràmetre.

Lògica d'Adam

Si un pes té gradients molt erràtics o petits, Adam augmenta el seu learning rate efectiu. Si són molt grans i constants, el redueix per no "passar-se de llarg". **Intuïció**: Si un gradient és molt variable o gran, dividim per la seva "mida" ($\sqrt{\mathbf{v}_t}$) per frenar. Si és petit i constant, l'accelerem.

Adam (I): Estimació de Moments de Gradients

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Adam tracta la seqüència de gradients $\nabla_{\theta}L(\theta_1), \nabla_{\theta}L(\theta_2), \dots, \nabla_{\theta}L(\theta_t)$ com un **senyal amb soroll**. Per netejar-lo, estima els moments al llarg del temps:

- **Primer Moment (m_t) $\approx$ Direcció:** Estima l'**esperança** del gradient. Filtra el soroll de l'SGD per mantenir una trajectòria estable.

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla_{\theta}L(\theta_t)$$

- **Segon Moment (v_t) $\approx$ Magnitud:** Estima el **moment cru de segon ordre** (relacionat amb la variància). Serveix per saber si estem en una zona de gradients gegants o molt petits.

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2)(\nabla_{\theta}L(\theta_t))^2$$

Amb $\beta_1, \beta_2 \in (0, 1)$.

Gradients com a variables aleatòries

Si pensem en els gradients com a variables aleatòries $\mathbf{g}_t \sim P(\nabla_{\theta}L)$, Adam manté estimacions recursives de $\mathbb{E}[\nabla_{\theta}L(\theta_t)]$ i $\mathbb{E}[(\nabla_{\theta}L(\theta_t))^2]$.

- $\beta = 0.99$: Memòria a llarg termini (molt pes al passat, poc al present).
- $\beta = 0.1$: Memòria a curt termini (oblida ràpid el passat).

Això permet a Adam “recordar” la forma del paisatge d'error sense guardar totes les dades.

Adam (II): Correcció del biaix (Bias Correction)

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Stochastic Gradient Descent

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

Com que inicialitzem els moments a zero ($\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$), els valors inicials de les mitjanes mòbils estan **esbiaixats cap al zero**.

$$t = 1 : \mathbf{m}_1 = (1 - \beta_1) \nabla_{\theta} L(\boldsymbol{\theta}_1)$$

$$t = 2 : \mathbf{m}_2 = \beta_1 \mathbf{m}_1 + (1 - \beta_1) \nabla_{\theta} L(\boldsymbol{\theta}_2) = \beta_1 (1 - \beta_1) \nabla_{\theta} L(\boldsymbol{\theta}_1) + (1 - \beta_1) \nabla_{\theta} L(\boldsymbol{\theta}_2)$$

L'estat a la iteració t és una combinació de tots els gradients anteriors:

$$\mathbf{m}_t = (1 - \beta_1) \sum_{r=1}^t \beta_1^{t-r} \nabla_{\theta} L(\boldsymbol{\theta}_r)$$

Desenvolupament general: Expandint la recursivitat per a qualsevol pas t , obtenim que l'esperança de l'estimador és:

$$\mathbb{E}[\mathbf{m}_t] = \mathbb{E}[\nabla_{\theta} L(\boldsymbol{\theta}_t)] \cdot (1 - \beta_1) \sum_{r=1}^t \beta_1^{t-r}$$

Aplicant la suma d'una progressió geomètrica:

$$\mathbb{E}[\mathbf{m}_t] = \mathbb{E}[\nabla_{\theta} L(\boldsymbol{\theta}_t)] \cdot (1 - \beta_1) \frac{1 - \beta_1^t}{1 - \beta_1} = \mathbb{E}[\nabla_{\theta} L(\boldsymbol{\theta}_t)] \cdot (1 - \beta_1^t)$$

Estimadors corregits (Unbiased)

Per compensar el l'last del zero inicial, dividim pel factor de biaix $(1 - \beta^t)$:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \text{i} \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

Adam (III): Actualització Adaptativa

Aprentatge
Automàtic II:
Fonaments
de l'Apre-
nentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

La regla final combina el **primer moment** (direcció suavitzada) i el **segon moment** (escala de la variabilitat) per moure els paràmetres:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}$$

Per què normalitzem amb la "magnitud" ($\sqrt{\hat{\mathbf{v}}_t}$)? Adam realitza un **escalat automàtic** de la taxa d'aprenentatge per a cada paràmetre:

- **Gradients consistents però petits:** Si un pes rep gradients minúsculs, $\hat{\mathbf{v}}_t$ serà petit. Això "amplifica" el pas, permetent que el model **acceleri en zones planes** (plateaus).
- **Gradients grans o sorollosos:** Si un pes oscil·la molt o rep gradients gegants, $\hat{\mathbf{v}}_t$ serà gran. Això "frena" el pas, actuant com un amortidor que **evita divergències** en zones de molta curvatura.

Conclusió: Un aprenentatge personalitzat

ϵ (típic 10^{-8}) evita la divisió per zero. A la pràctica, Adam és com tenir un **learning rate individual** per a cadascun dels milions de paràmetres de la xarxa, adaptant-se a la geometria local de cada dimensió.

Resum d'Adam

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Adam és l'algorisme més utilitzat avui dia. Combina dues idees:

- 1 **Momentum**: Mitjana mòbil dels gradients (m_t).
- 2 **RMSProp**: Mitjana mòbil del **quadrat** dels gradients (v_t) per normalitzar el pas.

Algorisme Adam

Per a cada paràmetre j , calculem els moments (típicament $\beta_1 = 0.9, \beta_2 = 0.999$):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\theta} L(\theta) \quad (\text{1er moment: mitjana})$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\theta} L(\theta))^2 \quad (\text{2on moment: variància})$$

Correcció de biaix (per a les primeres iteracions):

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

Actualització final:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

Quin optimitzador triar?

Aprentatge Automàtic II: Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Stochastic Gradient Descent

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

Algorisme	Idea Clau	Punt Fort / Escenari
SGD	Pendent local pur.	Simplicitat. Convergència teòrica a llarg termini.
Momentum	Acumula inèrcia (β).	Escapa de mínims locals i zones planes.
Nesterov	Mira el futur (<i>look-ahead</i>).	Evita l'oscil·lació en valls estretes i profundes.
Adam	Moments d'1er i 2on ordre.	L'estàndard. Molt robust; ideal per a dades sorolloses.

Recomanacions pràctiques

- **Adam** és el millor punt de partida (robust amb η per defecte).
- **SGD + Momentum** sovint troba millors mínims si s'ajusta amb molta cura (molt usat en recerca).

Atenció!

Cap optimitzador és "màgic". Si la xarxa no aprèn, sovint el problema és en la inicialització dels pesos o en una taxa d'aprenentatge (η) massa alta.

Hiperparàmetres d'entrenament

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Stochastic
Gradient Descent

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Més enllà de l'algorisme, el comportament del model depèn de variables que **no s'aprenen** durant el descens del gradient, sinó que hem de fixar:

- **Learning Rate (η):** El més crític. Si és molt alt, el model divergeix. Si és molt baix, l'entrenament és etern o es queda en zones planes.
- **Batch Size (B):** Controla el "soroll" de l'estimació del gradient.
- **Coeficients de Moment (β_1, β_2):** Determinen la "memòria" de l'optimitzador. Normalment s'usen els valors per defecte d'Adam (0.9 i 0.999).
- **Èpoques:** Quantes vegades la xarxa utilitza el dataset sencer.

La gran pregunta: Com sabem si són els correctes?

No podem triar hiperparàmetres a cegues. Per ajustar-los, necessitarem monitoritzar:

- **Loss Curves:** Evolució de l'error en el temps.
- **Overfitting:** Diferència entre l'error d'entrenament i el de test.

D'on surten els gradients? Backpropagation

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

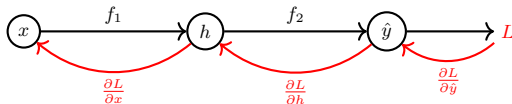
Fins ara hem assumit que coneixem $\nabla_{\theta} L$. En una xarxa amb milions de paràmetres, calcular cada derivada parcial analíticament seria impossible. L'algorisme de **Backpropagation** és una aplicació eficient de la **regla de la cadena** per calcular el gradient de la funció de pèrdua respecte a tots els pesos del model.

La Regla de la Cadena (recursiva)

Si tenim una composició de funcions $y = f(g(x))$, la derivada de y respecte a x és:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

En una xarxa neuronal, el gradient "flueix" des de la sortida (l'error) cap a l'entrada, multiplicant derivades locals capa per capa.



Backpropagation meme

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Exercici de Joguina: El camí d'una derivada

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

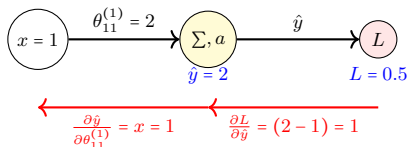
Backpropagation:
Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

Considerem: Input $x = 1$, pes $\theta_{11}^{(1)} = 2$, activació lineal $a(z) = z$, target $y = 1$. Loss: $L = \frac{1}{2}(\hat{y} - y)^2$.



Aplicació de la Regla de la Cadena

$$\frac{\partial L}{\partial \theta_{11}^{(1)}} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{Error final}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{a'(z)=1} \cdot \underbrace{\frac{\partial z}{\partial \theta_{11}^{(1)}}}_{x=1} = 1 \cdot 1 \cdot 1 = 1$$

- **Actualització** ($\eta = 0.1$): $\theta_{11}^{(1)} \leftarrow 2 - (0.1 \cdot 1) = 1.9$.
- Com que l'error és positiu i el pes és massa gran, la derivada ens diu que hem de reduir w .

Backpropagation: La Regla de la Cadena

En una xarxa de K capes, la sortida $\hat{y}$ és una **funció composta**:

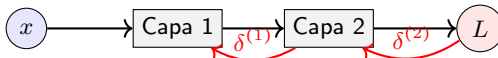
$$\hat{y} = \mathbf{h}^{(K)} = a^{(K)}(\Theta^{(K)}(\dots a^{(1)}(\Theta^{(1)}\mathbf{x}) \dots))$$

Principi Fonamental (Regla de la Cadena)

Si $y = f(u)$ i $u = g(x)$, aleshores $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$. A la xarxa, apliquem aquest principi de la sortida cap a l'entrada.

Backpropagation

- L'error es propaga des de la sortida cap a l'entrada.
- Cada capa rep un "missatge" (gradient) de la capa següent, el multiplica per la seva derivada local i el passa a la capa anterior.



Flux d'informació de l'error

Dimensions i Flux:

- $\mathbf{x} = \mathbf{h}^{(0)} \in \mathbb{R}^{D^{(0)} \times 1}$ (Input).
- $\Theta^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l-1)}}$ (Matriu de pesos de la capa l).
- $\mathbf{h}^{(l)} \in \mathbb{R}^{D^{(l)} \times 1}$ (Vector d'activacions de la capa l).

La Regla de la Cadena: De l'escalar a la matriu

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització

Per a una xarxa amb K capes, amb $D^{(l)}$ neurones de la capa l :

- **Matriu de pesos** $\Theta^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l-1)}}$: Connecta la unitat j (capa $l-1$) amb la unitat i (capa l).
- **Vector de biaixos** $\theta_0^{(l)} \in \mathbb{R}^{D^{(l)}}$: Paràmetres additius per a cada unitat i de la capa l .
- **Vector d'activacions** $\mathbf{h}^{(l-1)} \in \mathbb{R}^{D^{(l-1)}}$: Senyal que surt de la unitat j de la capa anterior.
- **Vector d'error** $\delta^{(l)} \in \mathbb{R}^{D^{(l)}}$: $\delta_i^{(l)} = \frac{\partial L}{\partial z_i^{(l)}}$ (sensibilitat respecte a la unitat i).

Regla de la Cadena per a pesos i biaixos individuals

$$\frac{\partial L}{\partial \theta_{ij}^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial \theta_{ij}^{(l)}} = \delta_i^{(l)} \cdot h_j^{(l-1)} \quad \bigg| \quad \frac{\partial L}{\partial \theta_{i0}^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial \theta_{i0}^{(l)}} = \delta_i^{(l)} \cdot 1$$

Expressió Matricial del Gradient

$$\nabla_{\Theta^{(l)}} L = \delta^{(l)} \cdot (\mathbf{h}^{(l-1)})^T \in \mathbb{R}^{D^{(l)} \times D^{(l-1)}} \quad \bigg| \quad \nabla_{\theta_0^{(l)}} L = \delta^{(l)} \in \mathbb{R}^{D^{(l)}}$$

Retropropagació de l'error ($\delta^{(l+1)} \rightarrow \delta^{(l)}$)

L'error d'una neurona j a la capa l és la suma ponderada de com ha afectat a totes les $D^{(l+1)}$ neurones de la capa següent:

Retropropagació

- Expressió per components:

$$\delta_j^{(l)} = \left(\sum_{k=1}^{D^{(l+1)}} \theta_{kj}^{(l+1)} \cdot \delta_k^{(l+1)} \right) \cdot a^{(l)'}(z_j^{(l)})$$

- Expressió Matricial:

$$\delta^{(l)} = \left((\Theta^{(l+1)})^T \delta^{(l+1)} \right) \odot a^{(l)'}(\mathbf{z}^{(l)})$$

Dimensions check:

- $(\Theta^{(l+1)})^T \in \mathbb{R}^{D^{(l)} \times D^{(l+1)}}$ propaga l'error cap enrere.
- $\delta^{(l)}$ té dimensió $\mathbb{R}^{D^{(l)}}$, que coincideix amb la mida del vector de biaixos $\theta_0^{(l)}$.
- Noti's que $\odot$ és el producte d'Hadamard:

$$\mathbf{A} \odot \mathbf{B} = \mathbf{C} \implies c_{ij} = a_{ij} \cdot b_{ij},$$

amb $\dim(\mathbf{A}) = \dim(\mathbf{B})$.

Resum: Les equacions del Backpropagation

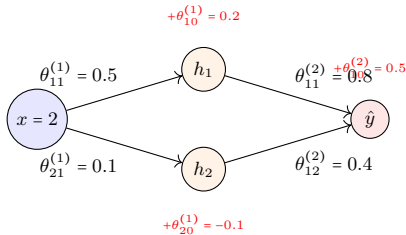
Per a una xarxa amb K capes, el càlcul de gradients es realitza en sentit invers ($l = K, K - 1, \dots, 1$):

Pas	Component (i, j)	Matricial (Capa l)
Error Sortida	$\delta_i^{(K)} = \frac{\partial L}{\partial \hat{y}_i} a^{(K)'}(z_i^{(K)})$	$\delta^{(K)} = \nabla_{\hat{\mathbf{y}}} L \odot a^{(K)'}(\mathbf{z}^{(K)})$
Retropropagar δ	$\delta_j^{(l)} = \left(\sum_k \theta_{kj}^{(l+1)} \delta_k^{(l+1)} \right) a^{(l)'}(z_j^{(l)})$	$\delta^{(l)} = ((\Theta^{(l+1)})^T \delta^{(l+1)}) \odot a^{(l)'}(\mathbf{z}^{(l)})$
∇ Pesos	$\frac{\partial L}{\partial \theta_{ij}^{(l)}} = \delta_i^{(l)} h_j^{(l-1)}$	$\nabla_{\Theta^{(l)}} L = \delta^{(l)} (\mathbf{h}^{(l-1)})^T$
∇ Biaixos	$\frac{\partial L}{\partial \theta_{i0}^{(l)}} = \delta_i^{(l)}$	$\nabla_{\theta_0^{(l)}} L = \delta^{(l)}$

- El gradient de la matriu de pesos $\nabla_{\Theta^{(l)}} L$ té dimensions $(D^{(l)} \times D^{(l-1)})$.
- El gradient del vector de biaixos $\nabla_{\theta_0^{(l)}} L$ té dimensions $(D^{(l)} \times 1)$.
- $\delta^{(l)}$ indica el "missatge d'error" que arriba a la capa l .
- $\mathbf{h}^{(l-1)}$ indica el valor que "venia" de la capa anterior. Recordatori: $\mathbf{h}^{(0)} = \mathbf{x}$

Escenari Numèric: Xarxa 1-2-1

Considerem $x = 2$, target $y = 1$ i activació $a(z) = \text{ReLU}$ (per l'exemple, usarem la part lineal on $a'(z) = 1$).



Forward Pass

- $\mathbf{z}^{(1)} = \Theta^{(1)}x + \theta_0^{(1)} \implies \mathbf{h}^{(1)} = a^{(1)}(\mathbf{z}^{(1)})$
 $h_1^{(1)} = (0.5 \cdot 2) + 0.2 = 1.2 \quad | \quad h_2^{(1)} = (0.1 \cdot 2) - 0.1 = 0.1$
- $\mathbf{z}^{(2)} = \Theta^{(2)}\mathbf{h}^{(1)} + \theta_0^{(2)} \implies \hat{y} = a^{(2)}(\mathbf{z}^{(2)})$
 $\hat{y} = (0.8 \cdot 1.2) + (0.4 \cdot 0.1) + 0.5 = 0.96 + 0.04 + 0.5 = 1.5$
- Error: $L = \frac{1}{2}(1.5 - 1)^2 = 0.125$

Backpropagation Pas a Pas: Capa de Sortida ($l = 2$)

Dades: $\hat{y} = 1.5, y = 1, h_1 = 1.2, h_2 = 0.1, a'(z) = 1$

Error local a la sortida $\delta^{(2)} \in \mathbb{R}^{1 \times 1}$

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z^{(2)}} = (\hat{y} - y) \cdot a'(z^{(2)}) = (1.5 - 1) \cdot 1 = \mathbf{0.5}$$

Gradients dels pesos $\nabla_{\Theta^{(2)}} L \in \mathbb{R}^{1 \times 2}$ (mateixes dimensions que $\Theta^{(2)}$)

$$\begin{aligned} \nabla_{\Theta^{(2)}} L &= \left(\frac{\partial L}{\partial \theta_{11}^{(2)}} \quad \frac{\partial L}{\partial \theta_{12}^{(2)}} \right) = \delta^{(2)} (\mathbf{h}^{(1)})^T = \left(\delta^{(2)} h_1^{(1)} \quad \delta^{(2)} h_2^{(1)} \right) = \\ &= (0.5 \cdot 1.2, 0.5 \cdot 0.1) = \mathbf{(0.6, 0.05)} \end{aligned}$$

Gradient del biaix $\theta_{10}^{(2)}$

$$\frac{\partial L}{\partial \theta_{10}^{(2)}} = \delta^{(2)} \implies \mathbf{0.5}$$

Backpropagation Pas a Pas: Capa Oculta ($l = 1$)

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

Gradient descent fitting models

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització

Dades: $x = 2, \delta^{(2)} = 0.5, \theta_{11}^{(2)} = 0.8, \theta_{12}^{(2)} = 0.4, a'(z) = 1$

Errors locals retropropagats $\delta^{(1)} \in \mathbb{R}^{2 \times 1}$

$$\begin{aligned}\delta^{(1)} &= \begin{pmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \end{pmatrix} = ((\Theta^{(2)})^T \delta^{(2)}) \odot a'(z^{(1)}) = \begin{pmatrix} \theta_{11}^{(2)} \\ \theta_{12}^{(2)} \end{pmatrix} (\delta^{(2)}) \odot \begin{pmatrix} a'(z_1^{(1)}) \\ a'(z_2^{(1)}) \end{pmatrix} = \\ &= \begin{pmatrix} \theta_{11}^{(2)} \delta^{(2)} a'(z_1^{(1)}) \\ \theta_{12}^{(2)} \delta^{(2)} a'(z_2^{(1)}) \end{pmatrix} = \begin{pmatrix} 0.8 \cdot 0.5 \cdot 1 \\ 0.4 \cdot 0.5 \cdot 1 \end{pmatrix} = \begin{pmatrix} \mathbf{0.4} \\ \mathbf{0.2} \end{pmatrix}\end{aligned}$$

Gradients dels pesos $\nabla_{\Theta^{(1)}} L \in \mathbb{R}^{2 \times 1}$ (mateixes dimensions que $\Theta^{(1)}$)

$$\nabla_{\Theta^{(1)}} L = \begin{pmatrix} \frac{\partial L}{\partial \theta_{11}^{(1)}} \\ \frac{\partial L}{\partial \theta_{21}^{(1)}} \end{pmatrix} = \delta^{(1)} (\mathbf{h}^0)^T = \begin{pmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \end{pmatrix} (x) = \begin{pmatrix} \delta_1^{(1)} \cdot x \\ \delta_2^{(1)} \cdot x \end{pmatrix} = \begin{pmatrix} 0.4 \cdot 2 \\ 0.2 \cdot 2 \end{pmatrix} = \begin{pmatrix} \mathbf{0.8} \\ \mathbf{0.4} \end{pmatrix}$$

Gradients dels biaixos $\theta_0^{(1)}$

$$\nabla_{\theta_0^{(1)}} L = \begin{pmatrix} \frac{\partial L}{\partial \theta_{10}^{(1)}} \\ \frac{\partial L}{\partial \theta_{20}^{(1)}} \end{pmatrix} = \delta^{(1)} = \begin{pmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \end{pmatrix} = \begin{pmatrix} \mathbf{0.4} \\ \mathbf{0.2} \end{pmatrix}$$

Actualització de Paràmetres (Gradient Descent)

Apliquem $\theta_{nou} = \theta_{vell} - \eta \cdot \frac{\partial L}{\partial \theta}$ amb $\eta = 0.1$.

Capa 2 (Sortida):

$$\nabla_{\Theta(2)} L = (0.6, 0.05), \quad \frac{\partial L}{\partial \theta_{10}^{(2)}} = 0.5$$

- $\theta_{11}^{(2)} : 0.8 - (0.1 \cdot 0.6) = \mathbf{0.74}$
- $\theta_{12}^{(2)} : 0.4 - (0.1 \cdot 0.05) = \mathbf{0.395}$
- $\theta_{10}^{(2)} : 0.5 - (0.1 \cdot 0.5) = \mathbf{0.45}$

Capa 1 (Oculta):

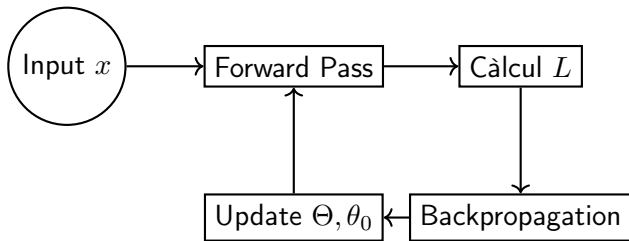
$$\nabla_{\Theta(1)} L = \begin{pmatrix} 0.8 \\ 0.4 \end{pmatrix}, \quad \nabla_{\theta_0^{(1)}} L = \begin{pmatrix} 0.4 \\ 0.2 \end{pmatrix}$$

- $\theta_{11}^{(1)} : 0.5 - (0.1 \cdot 0.8) = \mathbf{0.42}$
- $\theta_{21}^{(1)} : 0.1 - (0.1 \cdot 0.4) = \mathbf{0.06}$
- $\theta_{10}^{(1)} : 0.2 - (0.1 \cdot 0.4) = \mathbf{0.16}$
- $\theta_{20}^{(1)} : -0.1 - (0.1 \cdot 0.2) = \mathbf{-0.12}$

Conclusió de la iteració

Tots els paràmetres han disminuït. En el proper *forward pass*, la combinació de pesos i biaixos més petits farà que la $\hat{y}$ baixi del seu valor actual (1.5) per intentar arribar al target (1.0).

Resum: El cicle d'entrenament



- **Època (Epoch):** Una passada completa per tot el dataset.
- **Batch:** Subconjunt de dades usat per a una sola actualització (Stochastic Gradient Descent).

El perill de la simetria i l'explosió

No podem inicialitzar els pesos a **zero**.

- **Simetria:** Totes les neurones ocultes calcularien el mateix gradient i farien el mateix pas. Tindríem $D^{(1)}$ neurones idèntiques (inútil).

Problemes de variància:

- **Vanishing Gradient:** Si els pesos són molt petits, el gradient s'esvaeix en multiplicar-se capa rere capa. La xarxa no aprèn.
- **Exploding Gradient:** Si són molt grans, els valors s'escapen a l'infinit.

Solució: Inicialització de Xavier/Glorot (per a Tanh/Sigmoid)

Volem que la variància de les activacions sigui igual a la de l'entrada:

$$\Theta^{(l)} \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

Inicialització de He (per a ReLU)

$$\Theta^{(l)} \sim N\left(0, \frac{2}{n_{in}}\right)$$

Resum: L'algorisme complet d'entrenament

Aprentatge Automàtic II:
Fonaments de l'Aprentatge Automàtic i Deep Learning

Víctor Navas Portella

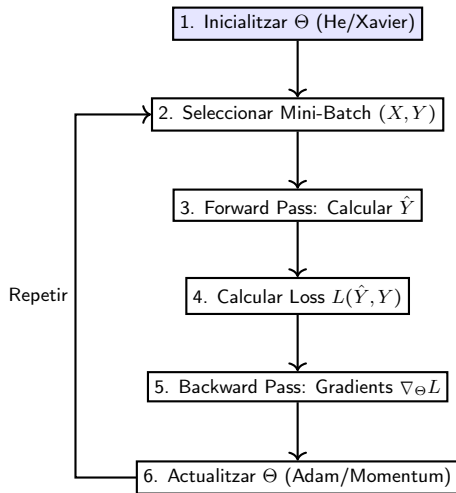
Gradient descent fitting models

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Basic recipe for deep learning

Aprenentatge Automàtic II:
Fonaments de l'Aprenentatge Automàtic i Deep Learning

Víctor Navas Portella

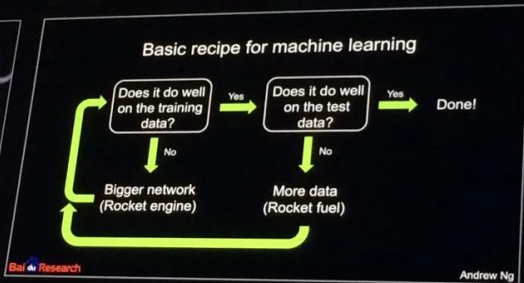
Gradient descent fitting models

Backpropagation: Càlcul de les derivades en una xarxa

Inicialització de paràmetres

Mètriques de performance

Regularització



Regularització

Aprenentatge
Automàtic II:
Fonaments
de l'Aprenentatge
Automàtic i
Deep
Learning

Víctor Navas
Portella

Gradient
descent
fitting
models

Backpropagation:
Càlcul de les
derivades en
una xarxa

Inicialització
de
paràmetres

Mètriques de
performance

Regularització